

MII MESH

An Offline-First Encrypted Messaging Application Built on a Bluetooth Low Energy Mesh Network

Framework	Flutter 3.47.0 / Dart 3.13.0 (Android-first)	Cryptography	Ed25519, X25519 sealed boxes, XChaCha20-Poly1305
Size	7,870 lines Dart, 468 lines Kotlin	Tests	20 of 20 passing; ARM64 APK builds

1. Abstract

Mainstream messaging depends entirely on centralised infrastructure: a message between two people in the same room is routed through a distant data centre, making the conversation dependent on cellular coverage, an internet subscription and a service provider. During disasters, network shutdowns and in crowded or remote places, that infrastructure is exactly what fails.

Mii Mesh removes the dependency. It forms a peer-to-peer network directly between nearby phones over **Bluetooth Low Energy**. Devices discover one another automatically, and messages travel either directly to a neighbour or hop-by-hop through intermediate phones acting as relays, so two people out of Bluetooth range can still communicate provided a chain of participants links them. No server, SIM card, internet connection, phone number or account is used at any stage.

Confidentiality is enforced cryptographically rather than by trusting relays. Each installation generates an Ed25519 identity on the device; private messages are sealed to the recipient's X25519 public key, so a relay forwards a packet it is mathematically unable to read, and every packet is signed so it cannot be altered undetected. Undeliverable messages are held in encrypted custody and delivered automatically when a route reappears. A bounded controlled-flooding algorithm with a seven-hop limit, duplicate suppression, per-link cooldowns and message expiry prevents the broadcast storms and routing loops that naive flooding produces. All local data is stored in a whole-file encrypted SQLite database.

2. Introduction

Cellular networks fail in three broad ways: *physical failure*, when storms or earthquakes destroy towers; *capacity failure*, when festivals or demonstrations saturate a cell; and *deliberate shutdown*. In each case the handsets remain fully functional computers with working short-range radios, yet conventional messaging applications become useless. A modern phone already carries everything needed for local communication — a BLE radio able to advertise and scan, ample storage, and a processor fast enough for public-key cryptography. What is missing is software that treats the phone as a self-sufficient network node rather than a client of a server.

2.1 Objectives

- Generate a cryptographic identity on-device, with no registration or personal information.
- Discover nearby participants automatically over BLE, with no manual pairing.
- Encrypt messages end-to-end so relays forward ciphertext they cannot read.
- Extend range through bounded multi-hop relaying with loop and duplicate protection.
- Retain undeliverable messages and deliver them automatically when a route reappears.
- Encrypt all data at rest, and present the network in plain language rather than networking terms.

Scope. The protocol, router, cryptography, persistence, application layer, native dual-role BLE radio and interface are implemented; all twenty automated tests pass and the Android package builds. A real authenticated Bluetooth exchange was demonstrated between a phone and a laptop companion. **Multi-hop relaying has not yet been tested across three or more physical handsets, and voice calling is not implemented.** This is an experimental prototype and has not been independently audited.

3. Literature Review and References

The design draws on delay-tolerant networking theory, published analyses of earlier Bluetooth mesh messengers, and standard cryptographic primitives. Each work below is followed by its relevance to this project.

- [1] Cerf, V. et al. *Delay-Tolerant Networking Architecture*. RFC 4838, IETF, 2007. Defines the store-carry-forward model for networks with no continuous end-to-end path. Mii Mesh's persistent packet store is a custody store in exactly this sense.
- [2] Scott, K. and Burleigh, S. *Bundle Protocol Specification*. RFC 5050, IETF, 2007. The concrete bundle and custody-transfer mechanics that informed the acknowledgement and retry design.
- [3] Vahdat, A. and Becker, D. *Epidemic Routing for Partially Connected Ad Hoc Networks*. Technical Report CS-2000-06, Duke University, 2000. Forward every message to every peer encountered: optimal delivery, unbounded cost. This is the algorithm Mii Mesh implements and then bounds.
- [4] Spyropoulos, T., Psounis, K. and Raghavendra, C. S. *Spray and Wait*. ACM SIGCOMM Workshop on Delay-Tolerant Networking, 2005. Limits epidemic routing by capping copies per message. Recorded as the next refinement of the routing layer.
- [5] Albrecht, M. R., Blasco, J., Jensen, R. B. and Mareková, L. *Mesh Messaging in Large-Scale Protests: Breaking Bridgefy*. CT-RSA, Springer, 2021. Found absent authentication and no forward secrecy in a deployed BLE mesh messenger. This analysis directly motivated the signed-packet and sealed-box design used here.
- [6] Bluetooth Special Interest Group. *Mesh Profile Specification v1.0*. 2017. The official BLE mesh profile, built for building automation over the advertising bearer. Phones cannot use it directly, which is why this project layers a mesh over standard GATT connections.
- [7] Bernstein, D. J., Duif, N., Lange, T., Schwabe, P. and Yang, B.-Y. *High-Speed High-Security Signatures*. Journal of Cryptographic Engineering 2(2), 2012. Ed25519, used for device identity and packet signatures.
- [8] Josefsson, S. and Liusvaara, I. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, IETF, 2017. The standardised form of Ed25519; its test vectors are used as a known-answer test in the suite.
- [9] Nir, Y. and Langley, A. *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439, IETF, 2018. The AEAD construction underlying the direct Bluetooth link.
- [10] Arciszewski, S. *XChaCha: eXtended-nonce ChaCha and AEAD_XChaCha20_Poly1305*. Internet-Draft, CFRG, IETF. The 192-bit nonce variant, which makes random nonce generation safe without a counter.
- [11] Perrin, T. *The Noise Protocol Framework*, revision 34. 2018. The standard basis for authenticated key exchange with forward secrecy — not implemented here, and identified as release-blocking.
- [12] The Briar Project. *Briar — Secure Messaging, Anywhere*. briarproject.org. A strong security model with forward secrecy, but requiring explicit contact pairing, whereas this project targets zero-configuration discovery.
- [13] Meshtastic Project. *Meshtastic — Off-Grid Mesh Communication*. meshtastic.org. Achieves kilometre-scale range using dedicated LoRa hardware; Mii Mesh deliberately targets unmodified consumer phones.
- [14] Frank Denis and contributors. *libsodium Documentation*. doc.libsodium.org. The cryptographic library providing all primitives used in the implementation.
- [15] Google LLC. *Bluetooth Permissions and Foreground Service Types*. Android Developers documentation. The platform rules governing runtime permissions and background execution for the native radio.

4. Algorithm

4.1 Packet Structure and Sealing

Every packet carries a signed header. The signature covers all header fields and the payload, so a relay can neither forge nor modify a packet; the hop counter sits outside the signed region because each relay must legitimately increment it. Private payloads are sealed to the recipient's X25519 key, so relays handle ciphertext alone.

```
MeshPacket {
  version      : 2
  messageId    : UUID      (dedup)
  senderId     : Ed25519 public key
  destinationId : key, or '*' for presence
  timestamp    : Unix milliseconds
  kind         : 'presence' | 'sealed'
  payload      : sealed ciphertext, <=4096
  maxHops      : 7
  --- signed above this line ---
  signature    : Ed25519 detached
  hops         : 0..7      (relay increments)
}

sendPrivate(peer, kind, body, ref):
  validate(kind, body, ref)
  plain <- [kind, body, ref]
  cipher <- box_seal(plain, peer.pubKey)
  pkt <- create(peer.id, 'sealed', cipher)
  pkt.sig <- ed25519_sign(pkt.header, sk)
  persist(pkt) // durable first
  showInUI(pkt) // never await radio
  for link in activeLinks:
    write(link, pkt)
```

4.2 Core Routing — Bounded Controlled Flooding

Each received packet is verified, then either consumed locally or relayed. Four independent mechanisms bound the flood: signature verification, a duplicate cache, a hop limit and expiry. Persistence always completes before a packet is forwarded or acknowledged, so a crash cannot lose a message whose receipt was already reported to its sender.

```
function onPacketReceived(link, bytes):
  packet <- decode(bytes) // rejects malformed input
  if not verifySignature(packet): discard; return
  if packet.sender == myId:      discard; return // own echo
  if packet.expired():           discard; return

  // 1. Presence: learn about a peer
  if packet.kind == 'presence':
    if known(sender) and known.key != publicKey: reject // pinned key changed
    if unknown or packet.hops <= known.hops:
      peers[sender] <- Peer(name, key, lastSeen=now, hops=packet.hops, via=link)

  // 2. Addressed to me: consume, never relay
  if packet.destination == myId:
    content <- sealOpen(packet.payload, mySecretKey)
    if content.kind == 'ack':
      remove content.reference from pendingQueue; mark delivered; return
    persist(content) // BEFORE acknowledging
    deliverToUI(content); send ack(packet.id) to sender; return

  // 3. Not mine: relay under strict bounds
  if seenCache.contains(packet.id): discard; return // duplicate
  seenCache.insert(packet.id, expiry = packet.expires)
  if packet.hops >= MAX_HOPS (7): discard; return // TTL spent
  relayed <- packet.withHops(packet.hops + 1)
  if packet.kind == 'sealed': pendingQueue.add(relayed) // accept custody
  for l in activeLinks where l != link: // never echo to source
    write(l, relayed); lastSent[l, packet.id] <- now

function flush(): // every 5 seconds
  for packet in pendingQueue:
    if packet.expired(): remove from queue and storage; continue
  for link in activeLinks:
    if now - lastSent[link, packet.id] < COOLDOWN (15 s): continue
    write(link, packet); lastSent[link, packet.id] <- now
```

4.3 Bounds and Reachability

Bound	Value	Internal condition	Shown to the user
Maximum hops (TTL)	7	hops = 0, seen < 60 s	Nearby
Duplicate cache	4,000 IDs	hops > 0, seen < 60 s	Via mesh
Message lifetime	24 hours	Last seen > 60 s ago	Recently seen
Presence validity	60 seconds	Adapter disabled	Bluetooth is off
Custody queue	500 packets	Scanning, no peers	Searching for nearby people...
Peer table	256 peers	Write failed, retrying	Connection interrupted — retrying...
Per-link cooldown	15 seconds		
Maximum payload	4,096 bytes		
Chunks per packet	512		

Link-layer framing splits each packet into indexed chunks, rejecting out-of-order indices, implausible totals and transfers stalled beyond ten seconds, with reassembly bounded at four kilobytes. Internal connection state never reaches the interface as networking vocabulary; GATT, MTU, RSSI and UUID appear only in a developer diagnostics screen.

5. Screenshots

All images are genuine rendered output of the application, captured from its own widget test harness at real device resolutions. Peer entries in Figures 4 and 7 are seeded test data, since populating them from live radio traffic requires several physical handsets.

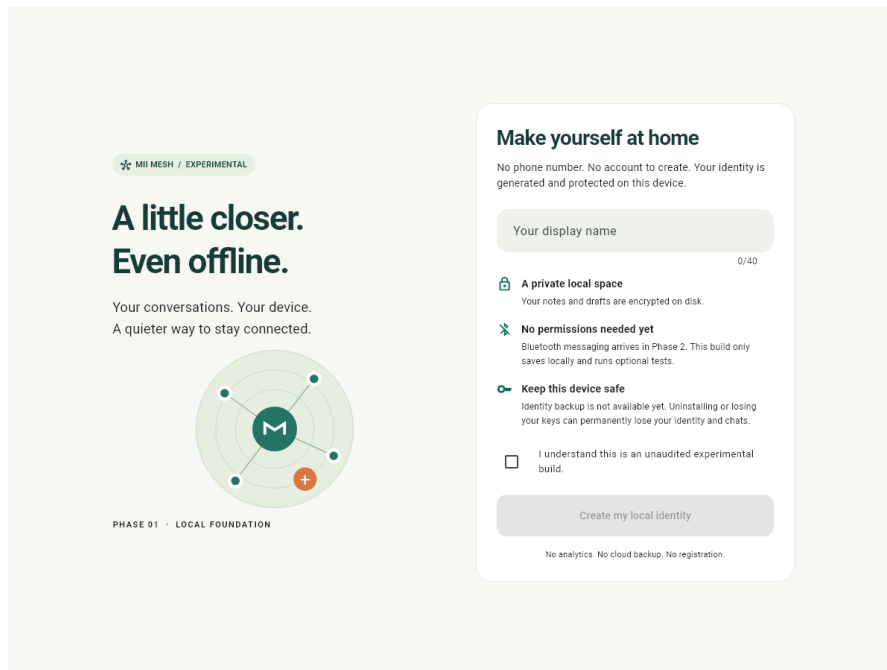


Figure 1 — Onboarding. The identity is generated on the device; no account or phone number is requested, and the experimental status must be acknowledged.

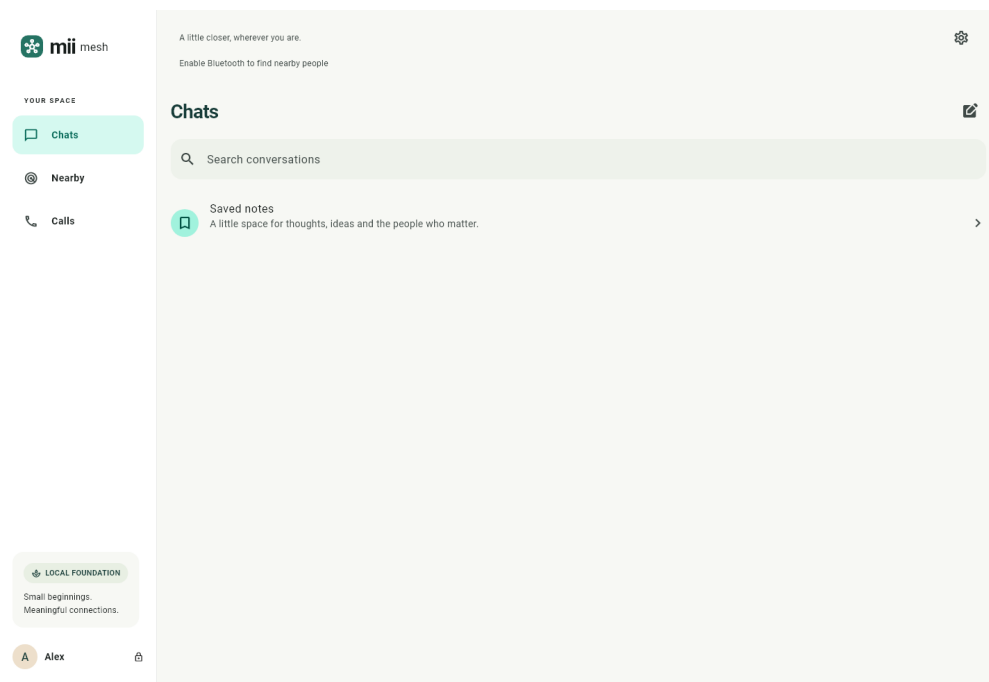


Figure 2 — Chats on a wide layout: three-section navigation, plain-language mesh status, and settings reduced to one icon.

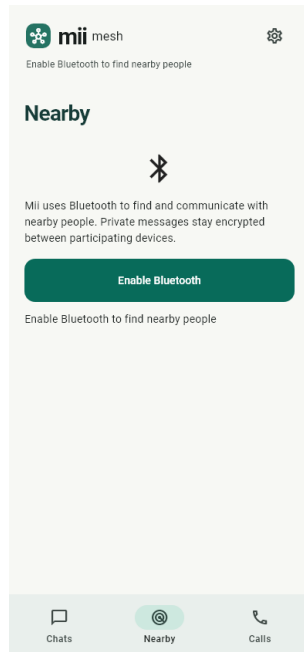


Figure 3 — First use: the purpose of Bluetooth access is explained before any system dialog.

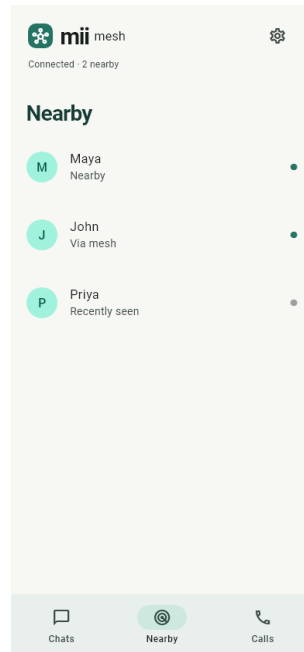


Figure 4 — Peers discovered, showing the three reachability states in plain language.

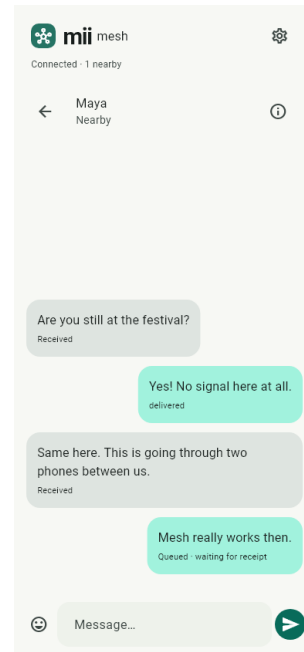


Figure 5 — A conversation showing delivery states: received, delivered and queued.

Discovery is automatic — there is no scan button and no manual pairing. Each entry states how that person is reachable: **Nearby** for a direct link, **Via mesh** when the route passes through intermediate devices, and **Recently seen** once presence lapses. Messages appear immediately on sending while encryption and routing proceed in the background; stickers and reactions travel as identifiers rather than images.

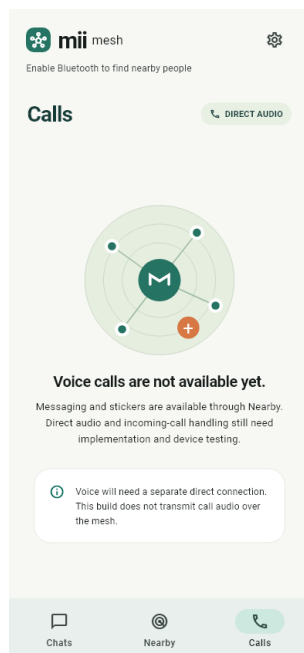


Figure 6 — Calls states plainly that voice calling is not implemented.

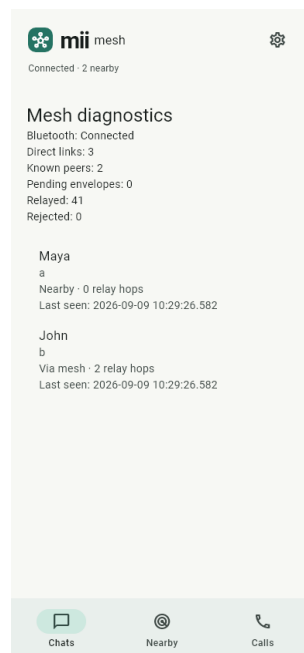


Figure 7 — Diagnostics: links, relay and rejection counters, hop distance, last seen.

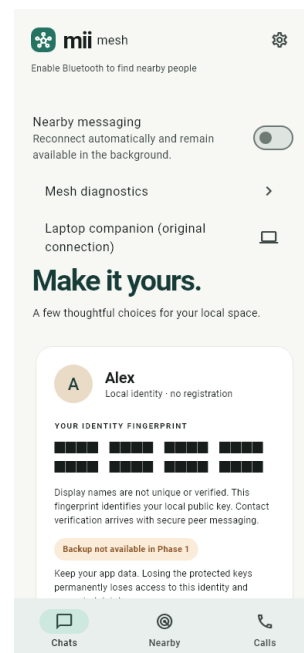


Figure 8 — Settings: identity fingerprint, appearance and mesh controls.

Rather than offering a control that would fail, the Calls section explains why voice is absent: continuous audio cannot be carried hop-by-hop across a BLE mesh and needs a separate direct connection. All networking detail is confined to the diagnostics screen, which an ordinary user never needs to open.

6. Conclusion

The project set out to show that useful, private messaging is possible between nearby smartphones with no server, no internet connection and no account. That has been achieved to the point where only hardware validation remains.

A complete mesh stack is implemented: a signed packet format, anonymous sealed-box encryption to the recipient's public key, a bounded controlled-flooding router with duplicate suppression, hop limiting, expiry and store-and-forward custody, a service binding that router to durable storage, and a native Android radio operating simultaneously as BLE peripheral and central with chunked framing for packets exceeding a single write. All twenty automated tests pass and the ARM64 package builds.

The decision that matters most is the separation of routing from confidentiality. Because payloads are sealed to the recipient's X25519 key rather than merely link-encrypted, a relay performs its forwarding role while remaining cryptographically unable to read what it carries. This is what makes relaying by untrusted strangers acceptable at all, and it addresses the weaknesses that published analysis found in earlier BLE mesh messengers [5].

Comparable attention went into bounding the system. Naive flooding destroys mesh networks through broadcast storms, routing loops and unbounded memory growth; eleven independent bounds constrain the algorithm so its worst case stays predictable, and the parser was hardened against hostile input and confirmed with 1,500 randomised malformed packets.

The honest boundary of this work is hardware validation. A real authenticated single-hop Bluetooth exchange has been demonstrated and the application runs and recovers correctly on a physical Android handset, but multi-hop relaying between three or more phones — the definitive test of a mesh — has not been performed. The routing algorithm is implemented, reasoned about and unit-tested against simulated links; it is not yet field-proven, and describing it otherwise would overstate the evidence. Within that boundary the project succeeds: an ordinary smartphone can act as a self-sufficient, privacy-preserving network node using only the radio it already carries.

7. Future Scope

7.1 Immediate

- **Multi-device validation.** Exercise $A \leftrightarrow B$, $A \leftrightarrow B \leftrightarrow C$ and $A \leftrightarrow B \leftrightarrow C \leftrightarrow D$ topologies, remove an intermediate node to confirm an alternative route is found, and verify store-and-forward across real disconnections and movement.
- **Radio tuning.** Characterise concurrent advertising and scanning across Android vendors, negotiate larger MTUs, and tune intervals and back-off to avoid connection storms.
- **iOS implementation.** The Dart layer is portable; the native radio and background behaviour must be written against Apple's stricter execution model.

7.2 Security Hardening (Release-Blocking)

- **A Noise handshake** [11] with X25519, HKDF-SHA-256, transcript binding and session rotation, giving mutual authentication and **forward secrecy**, which the current design lacks.
- **Contact verification** by fingerprint or QR exchange, with explicit key-change warnings.
- **Persistent replay protection** across restarts, anti-amplification limits and per-peer quotas.
- **Identity backup and recovery**, absent today — losing the device loses the identity.
- **An independent security audit** and a full dependency and licence review.

7.3 Features and Research

- **Voice.** Call signalling can travel over the mesh, but continuous audio must not: the design negotiates over Bluetooth then establishes a direct high-bandwidth link for audio. Where none is possible, low-bitrate push-to-talk through the mesh is the honest fallback.
- **Media transfer** with per-attachment keys, chunk hashing and resumable transfer; **encrypted group channels** with key rotation on membership change; **priority queues** so a large transfer never delays a short message.
- **Adaptive power management** with active, balanced and idle duty cycles.

- **Research:** Spray-and-Wait copy limiting [4], hybrid Wi-Fi Direct transport for bulk data, traffic-analysis resistance through padding and randomised delays, and scalability studies as participant density grows.

Closing note. Mii Mesh is an experimental, unaudited research prototype. It should not be relied upon where a communication failure or disclosure of message content would cause harm. Its value is in demonstrating a working architecture and documenting precisely what remains to be verified.